CA2_CNN_Template_02

October 15, 2026

1 Compulsory Assignment 2: Convolutional neural networks

Please fill out the group name, number, members and optionally the name below.

Group number: 20
Group member 1: Oskar Støre
Group member 2: Magnus Lundetræ Fidje
Group member 3: Håkon Skramstad
Group name (optional):

2 Assignment Submission

To complete this assignment answer the relevant questions in this notebook and write the code required to implement the relevant models. The assignment is submitted by handing in this notebook as an .ipynb file and as a .pdf file.

3 Introduction

This assignment will start with classifying handwritten digits from the MNIST dataset, used in the voluntary assignment and the first compulsory assignment. The second part of this task will revolve around classifying the open-source Fashion-MNIST.

3.1 Fashion-MNIST

The Fashion-MNIST is a dataset created from Zalando's article images(https://github.com/zalandoresearch/fashion-mnist/blob/master/README.md). The dataset consists of 70,000 labels images in 28x28 grayscale image labeled from 0 to 10. The goal of the Fashion-MNIST to work as a harder version of the handletter dataset MNIST. The Github repo for the dataset describe the dataset to be: - "intend Fashion-MNIST to serve as a direct drop-in replacement for the original MNIST dataset for benchmarking machine learning algorithms."

4 LeNet-5 Architecture Overview

LeNet-5 (LeCun et al., 1998) is a foundational Convolutional Neural Network (CNN) designed for handwritten digit classification (MNIST). It established the classic deep learning workflow: alternating **Convolution** and **Pooling** layers to extract spatial features, followed by **Dense** layers for classification.

4.1 1. Network Pipeline

Input Image ⟶ Conv1 ⟶ Pool1 ⟶ Conv2 ⟶ Pool2 ⟶ Flatten ⟶ FC1 ⟶ FC2 ⟶ Output

4.2 2. Layer Specification Table

Layer	Type	Configuration / Parameters	Activation
C1	Conv2D	6 filters (5×5), Padding='same'	`tanh`
S2	AvgPooling	Window: 2×2, Stride: 2	—
C3	Conv2D	16 filters (5×5), Padding='valid'	`tanh`
S4	AvgPooling	Window: 2×2, Stride: 2	—
Flatten	Reshape	Converts feature maps to 1D vector	—
F5	Dense	120 neurons	`tanh`
F6	Dense	84 neurons	`tanh`
Output	Dense	10 neurons (Classes 0–9)	`softmax`

4.3 3. Key Concepts for Students

- **Hierarchical Feature Extraction:** Early layers extract simple visual features (edges and lines), while deeper layers combine them into complex shapes (loops and intersections).
- **Spatial Invariance:** Subsampling (Pooling) reduces feature map resolution, making the model resilient to minor shifts or rotations in input images.
- **Evolution to Modern CNNs:**
  - **Activations:** Replaced historical `tanh` with `ReLU` to eliminate vanishing gradients.
  - **Pooling:** Replaced `AveragePooling` with `MaxPooling` for stronger feature selection.
  - **Normalization:** Added `BatchNorm` after convolution operations to accelerate convergence and stabilize training. ## Assignment structure

1. Part 1: Implementing LeNet for classifying MNIST.
2. Part 2: Designing your own CNN for classifying Fashion-MNIST

```
prediction = model.predict(X_test)
flat_prediction = np.argmax(prediction, axis=1) # Flatten softmax predictions
submissionDF = pd.DataFrame()
submissionDF['ID'] = range(len(flat_prediction)) # The submission csv file must have an row in
submissionDF['Prediction'] = flat_prediction
submissionDF.to_csv('submission.csv', index=False) # Remember to store the dataframe to csv wi
```

4.4 Library imports

[42]:
```
# Feel free to add or remove libraries as you want
import time

import numpy as np
import pandas as pd
import h5py

import matplotlib.pyplot as plt
import seaborn as sns

import tensorflow.keras as ks

SEED = 458
RNG = np.random.default_rng(SEED) # Random number generator

from utilities import *

import tensorflow as tf

# import confusion matrix
from sklearn.metrics import confusion_matrix

import seaborn as sns
```

5 Part 1: CNN for classifying the MNIST dataset

5.1 Loading MNIST

[43]:
```
datasets = load_mnist(verbose=0)
X_train, y_train = datasets['X_train'], datasets['y_train']
X_val,   y_val   = datasets['X_val'],   datasets['y_val']
X_test,  y_test  = datasets['X_test'],  datasets['y_test']

X_train = np.concatenate([X_train, X_val], axis=0)
y_train = np.concatenate([y_train, y_val], axis=0).astype('int32')

del datasets, X_val, y_val # Good to reduce uneccesary RAM usage
```

5.2 Preprocessing

[44]:
```
# Reshape data to account for color channel
X_train = np.expand_dims(X_train, -1)
X_test  = np.expand_dims(X_test, -1)
```

```
# Normalizing input between [0,1]
X_train = X_train.astype("float32")/np.max(X_train)
X_test  = X_test.astype("float32")/np.max(X_test)

# Converting targets from numbers to categorical format
y_train = ks.utils.to_categorical(y_train, len(np.unique(y_train)))
y_test  = ks.utils.to_categorical(y_test, len(np.unique(y_test)))
```

5.3 Task 1.1: Build a CNN network with the LeNet5 architecture

Implement LeNet5 architecture according to the previous specifications:

Compile the network with the

- `tf.keras.losses.CategoricalCrossentropy` loss function
- the `adam` optimizer
- with the `accuracy` metric and (your own implementation of the) F1-score metric.

[45]:
```
# build F1 score
def f1score(y_true, y_pred):
    y_pred = tf.one_hot(tf.argmax(y_pred, axis=1), 10)
    tp = tf.reduce_sum(y_true * y_pred, axis=0)
    fp = tf.reduce_sum((1 - y_true) * y_pred, axis=0)
    fn = tf.reduce_sum(y_true * (1 - y_pred), axis=0)
    f1 = 2 * tp / (2 * tp + fp + fn + 1e-7)
    return tf.reduce_mean(f1)
```

[46]:
```
model = ks.Sequential([
    ks.layers.Conv2D(filters=6, kernel_size=(5, 5), padding='same',␣
 ↪name='Conv1', activation='tanh'),
    ks.layers.AvgPool2D(pool_size=(2, 2), strides=(2, 2), name='Pool1'),
    ks.layers.Conv2D(filters=16, kernel_size=(5, 5), padding='valid',␣
 ↪name='Conv2', activation='tanh'),
    ks.layers.AvgPool2D(pool_size=(2, 2), strides=(2, 2), name='Pool2'),
    ks.layers.Flatten(),  # flatten before giving to nn
    ks.layers.Dense(units=120, name='FC1', activation='tanh'),
    ks.layers.Dense(units=84, name='FC2', activation='tanh'),
    ks.layers.Dense(units=10, name='Output', activation='softmax'),
])
```

5.3.1 Task 1.1.2 Train network

Train the network with a * batch size of 64 samples * for 20 epochs * 1/8 validation split

[47]:
```
# don't need to specify batch size
model.build(input_shape=(None, 28, 28, 1))
```

```
model.compile(
    optimizer=ks.optimizers.Adam(learning_rate=0.001),
    loss= ks.losses.CategoricalCrossentropy(),
    metrics= ['accuracy', f1score]
)

model.summary()
```

Model: "sequential_9"

Layer (type)	Output Shape	Param #
Conv1 (Conv2D)	(None, 28, 28, 6)	156
Pool1 (AveragePooling2D)	(None, 14, 14, 6)	0
Conv2 (Conv2D)	(None, 10, 10, 16)	2,416
Pool2 (AveragePooling2D)	(None, 5, 5, 16)	0
flatten_9 (Flatten)	(None, 400)	0
FC1 (Dense)	(None, 120)	48,120
FC2 (Dense)	(None, 84)	10,164
Output (Dense)	(None, 10)	850

Total params: 61,706 (241.04 KB)

Trainable params: 61,706 (241.04 KB)

Non-trainable params: 0 (0.00 B)

[48]:
```
history = model.fit(X_train, y_train, batch_size=64, epochs=20,␣
↪validation_split=1/8)
```

```
Epoch 1/20
821/821             4s 4ms/step -
accuracy: 0.9131 - f1score: 0.9018 - loss: 0.2959 - val_accuracy: 0.9504 -
val_f1score: 0.9425 - val_loss: 0.1549
Epoch 2/20
```

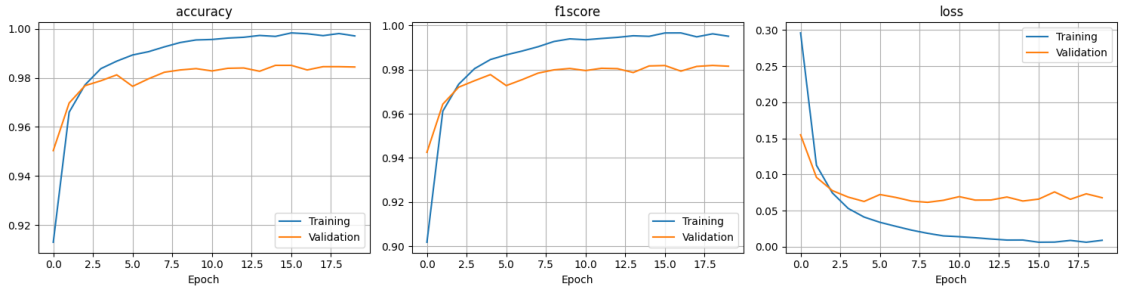
```
821/821            3s 4ms/step -
accuracy: 0.9660 - f1score: 0.9612 - loss: 0.1127 - val_accuracy: 0.9697 -
val_f1score: 0.9644 - val_loss: 0.0960
Epoch 3/20
821/821            3s 4ms/step -
accuracy: 0.9771 - f1score: 0.9734 - loss: 0.0745 - val_accuracy: 0.9768 -
val_f1score: 0.9720 - val_loss: 0.0777
Epoch 4/20
821/821            3s 4ms/step -
accuracy: 0.9837 - f1score: 0.9804 - loss: 0.0530 - val_accuracy: 0.9788 -
val_f1score: 0.9750 - val_loss: 0.0687
Epoch 5/20
821/821            3s 3ms/step -
accuracy: 0.9868 - f1score: 0.9846 - loss: 0.0411 - val_accuracy: 0.9812 -
val_f1score: 0.9777 - val_loss: 0.0626
Epoch 6/20
821/821            3s 4ms/step -
accuracy: 0.9893 - f1score: 0.9867 - loss: 0.0338 - val_accuracy: 0.9765 -
val_f1score: 0.9728 - val_loss: 0.0722
Epoch 7/20
821/821            3s 4ms/step -
accuracy: 0.9906 - f1score: 0.9884 - loss: 0.0282 - val_accuracy: 0.9796 -
val_f1score: 0.9755 - val_loss: 0.0682
Epoch 8/20
821/821            3s 4ms/step -
accuracy: 0.9926 - f1score: 0.9904 - loss: 0.0229 - val_accuracy: 0.9823 -
val_f1score: 0.9784 - val_loss: 0.0632
Epoch 9/20
821/821            3s 4ms/step -
accuracy: 0.9944 - f1score: 0.9927 - loss: 0.0186 - val_accuracy: 0.9832 -
val_f1score: 0.9799 - val_loss: 0.0614
Epoch 10/20
821/821            3s 4ms/step -
accuracy: 0.9954 - f1score: 0.9939 - loss: 0.0150 - val_accuracy: 0.9837 -
val_f1score: 0.9805 - val_loss: 0.0643
Epoch 11/20
821/821            3s 4ms/step -
accuracy: 0.9956 - f1score: 0.9935 - loss: 0.0139 - val_accuracy: 0.9828 -
val_f1score: 0.9796 - val_loss: 0.0693
Epoch 12/20
821/821            3s 4ms/step -
accuracy: 0.9962 - f1score: 0.9941 - loss: 0.0124 - val_accuracy: 0.9839 -
val_f1score: 0.9806 - val_loss: 0.0645
Epoch 13/20
821/821            3s 4ms/step -
accuracy: 0.9965 - f1score: 0.9946 - loss: 0.0107 - val_accuracy: 0.9840 -
val_f1score: 0.9804 - val_loss: 0.0647
Epoch 14/20
```

```
821/821             4s 4ms/step -
accuracy: 0.9972 - f1score: 0.9953 - loss: 0.0092 - val_accuracy: 0.9827 -
val_f1score: 0.9787 - val_loss: 0.0688
Epoch 15/20
821/821             4s 4ms/step -
accuracy: 0.9969 - f1score: 0.9951 - loss: 0.0093 - val_accuracy: 0.9851 -
val_f1score: 0.9817 - val_loss: 0.0633
Epoch 16/20
821/821             4s 5ms/step -
accuracy: 0.9983 - f1score: 0.9966 - loss: 0.0062 - val_accuracy: 0.9851 -
val_f1score: 0.9818 - val_loss: 0.0659
Epoch 17/20
821/821             3s 4ms/step -
accuracy: 0.9979 - f1score: 0.9966 - loss: 0.0064 - val_accuracy: 0.9832 -
val_f1score: 0.9793 - val_loss: 0.0759
Epoch 18/20
821/821             4s 4ms/step -
accuracy: 0.9972 - f1score: 0.9948 - loss: 0.0087 - val_accuracy: 0.9845 -
val_f1score: 0.9814 - val_loss: 0.0656
Epoch 19/20
821/821             4s 4ms/step -
accuracy: 0.9980 - f1score: 0.9962 - loss: 0.0062 - val_accuracy: 0.9845 -
val_f1score: 0.9819 - val_loss: 0.0732
Epoch 20/20
821/821             4s 5ms/step -
accuracy: 0.9970 - f1score: 0.9951 - loss: 0.0089 - val_accuracy: 0.9844 -
val_f1score: 0.9815 - val_loss: 0.0678
```

5.4 Task 1.2 Evaluaiton

5.4.1 Task 1.2.1 Plot training history and evaluate on test dataset

[49]:
```
plot_training_history(history)
```



5.4.2 Task 1.2.2 Evaluate on the test dataset

[50]:
```
evaluate = model.evaluate(X_test,y_test)
```

```
313/313             0s 2ms/step -
accuracy: 0.9867 - f1score: 0.9595 - loss: 0.0498
```

5.4.3 Task 2.2.3 Create a confution matrix for both traing and testing data

- Does the test data and train data predikt the same items wrong?

[51]:
```
# creating confusion matrix for  test data
y_prob_test = model.predict(X_test)

# have to change to give one prediction for each picture
y_pred_test = np.argmax(y_prob_test,axis=1)
y_test_conf = np.argmax(y_test,axis=1)

cf_test = confusion_matrix(y_test_conf,y_pred_test, normalize='true')

# creating confusion matrix for training data
y_prob_train = model.predict(X_train)

# same as with test, changing to prediction
y_pred_train = np.argmax(y_prob_train, axis=1)
y_train_conf = np.argmax(y_train, axis=1)

cf_train = confusion_matrix(y_train_conf, y_pred_train, normalize='true')

# plotting
plt.figure(figsize=(12,10))

plt.subplot(2,1,1)
sns.heatmap(cf_test, annot=True, fmt='.1%', cmap='viridis')
plt.xlabel('Predictied class')
plt.ylabel('Actual class')
plt.title('Confusion matrix for test data')

plt.subplot(2,1,2)
sns.heatmap(cf_train, annot=True, fmt='.1%', cmap='viridis')
plt.xlabel('Predictied class')
plt.ylabel('Actual class')
plt.title('Confusion matrix for training data')

plt.tight_layout()

plt.show()
```

```
313/313             0s 1ms/step
```

```
1875/1875                    2s 1ms/step
```



Both the training and test data have very high accuracy, so they are not easy to compare. In both sets, 4 is sometimes misclassified as 9 (0.4% in training and 0.8% in test). On the test set we also see that 7 is sometimes misclassified as 2 (0.8%), which does not occur in the training data.

5.5 Task 2.3: MNIST discussion

Task 2.3.1: What is overfitting and how does it occur during training of the LeNet-5 model

Task 2.3.2: What is ReLU and Leaky ReLU, what is their derivative and how does their derivative solve the vanishing gradient problem?

Task 2.3.3: Calculate the numbers of weight and the output shape for the LeNet-5 at each layer, Show calculation

Answer 2.3.1: Overfitting is when the model fits the training data too closely and does not generalize well, so it performs worse when tested on new, unseen data. In the LeNet-5 model we can see this starting to happen after roughly 5 epochs: the validation loss flattens out while the training loss keeps decreasing. The validation accuracy also starts to flatten out.

Answer 2.3.2: ReLU and Leaky ReLU are two different activation functions.

ReLU ReLU outputs zero for negative input and passes positive input through unchanged. Its derivative is 0 when the input is less than zero and 1 when the input is greater than zero. Since the derivative is a constant 1 for positive input, the gradient does not shrink as it is propagated back through the layers. This reduces the vanishing gradient problem during backpropagation.

Leaky ReLU Leaky ReLU is the same as ReLU for positive input, but for negative input it has a small positive slope, alpha, which gives a small negative output. The derivative of Leaky ReLU is therefore 1 for positive input and alpha for negative input. Because the gradient is never exactly zero, neurons cannot "die" the way they can with ReLU. The network can still learn from negative input, and the gradient does not vanish.

Answer 2.3.3:

Input: $28 \times 28 \times 1$

5.5.1 Conv1: Conv2D (6 filters, 5×5, padding = 'same')

- $n = 28$, $m = 5$, $s = 1$, $K = 6$, $C_{in} = 1$
- Padding 'same': $p = \frac{m-1}{2} = \frac{5-1}{2} = 2$

$$o = \left\lfloor \frac{28 + 2 \cdot 2 - 5}{1} \right\rfloor + 1 = 27 + 1 = 28$$

$$\text{params} = (5 \cdot 5 \cdot 1 + 1) \cdot 6 = 26 \cdot 6 = 156$$

Output shape: $28 \times 28 \times 6$, **Weights:** 156

5.5.2 Pool1: AvgPooling (2×2, stride 2)

- $n = 28$, $m = 2$, $s = 2$

$$o = \left\lfloor \frac{28 - 2}{2} \right\rfloor + 1 = 13 + 1 = 14$$

Output shape: $14 \times 14 \times 6$, **Weights:** 0

5.5.3 Conv2: Conv2D (16 filters, 5×5, padding = 'valid')

- $n = 14$, $m = 5$, $s = 1$, $K = 16$, $C_{in} = 6$
- Padding 'valid': $p = 0$

$$o = \left\lfloor \frac{14 + 2 \cdot 0 - 5}{1} \right\rfloor + 1 = 9 + 1 = 10$$

$$\text{params} = (5 \cdot 5 \cdot 6 + 1) \cdot 16 = 151 \cdot 16 = 2416$$

Output shape: $10 \times 10 \times 16$, **Weights:** 2416

5.5.4 Pool2: AvgPooling (2×2, stride 2)

- $n = 10$, $m = 2$, \$s = 2

$$o = \left\lfloor \frac{10 - 2}{2} \right\rfloor + 1 = 4 + 1 = 5$$

Output shape: $5 \times 5 \times 16$, **Weights:** 0

5.5.5 Flatten

$$5 \cdot 5 \cdot 16 = 400$$

Output shape: 400, **Weights:** 0

5.5.6 FC1: Dense (120 neurons)

- $N_{in} = 400$, $N_{out} = 120$

$$\text{params} = (400 + 1) \cdot 120 = 48120$$

Output shape: 120, **Weights:** 48120

5.5.7 FC2: Dense (84 neurons)

- $N_{in} = 120$, $N_{out} = 84$

$$\text{params} = (120 + 1) \cdot 84 = 10164$$

Output shape: 84, **Weights:** 10164

5.5.8 Output: Dense (10 neurons)

- $N_{in} = 84$, $N_{out} = 10$

$$\text{params} = (84 + 1) \cdot 10 = 850$$

Output shape: 10, **Weights:** 850

6 Task 2: CNN for classifying the Fashion-MNIST dataset

In this task you shall implement a CNN model, and train it to classify the images in the Fashion-MNIST dataset.

6.1 Importing Fashion-MNIST

- Filename: `Fashion_MNIST.h5`

[52]:
```
# Change this path
#path = r"C:\Users\alfa\OneDrive - Norwegian University of Life␣
 ↪Sciences\Desktop\AML_NMBU\DAT300\DAT300_2025\CA2_CNN_Template-1\Template\Fashion_MNIST.
 ↪h5"
path = r"/home/skramstad/Documents/DAT300/Assignments/CA2/Fashion_MNIST.h5"
labels = {
    0: "T-shirt/top",
    1: "Trouser",
    2: "Pullover",
    3: "Dress",
    4: "Coat",
    5: "Sandal",
    6: "Shirt",
    7: "Sneaker",
    8: "Bag",
    9: "Ankle boot"
}
```

[53]:
```
# 2. Use 'path' variable here
with h5py.File(path, 'r') as f:
    print('Datasets in file:', list(f.keys()))
    X_train = np.asarray(f['train_images'])
    y_train = np.asarray(f['train_labels'])
    X_test  = np.asarray(f['test_images'])
    print('Nr. train images: %i' % (X_train.shape[0]))
    print('Nr. test images: %i' % (X_test.shape[0]))
```

```
Datasets in file: ['test_images', 'train_images', 'train_labels']
Nr. train images: 59500
Nr. test images: 10500
```

6.2 Task 2.1: Preprocess the data

Preprocess the data as you see fit

[54]:
```
# printing to see shape first
print(f'X_train shape: {X_train.shape} \n'
      f'X_test shape: {X_test.shape} \n'
      f'y_train shape: {y_train.shape}')
```

```
X_train shape: (59500, 28, 28)
X_test shape: (10500, 28, 28)
y_train shape: (59500,)
```

[55]:
```
# adding a fourth dimension
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], X_train.
 ↪shape[2], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], X_test.shape[2],␣
 ↪1))

y_train = y_train[:, np.newaxis]
```

[56]:
```
# normalize inputs from 0-255 to 0.0 - 1.0
X_train = X_train.astype("float32")/np.max(X_train)
X_test = X_test.astype("float32")/np.max(X_test)
```

[57]:
```
# Converting targets from numbers to categorical format
y_train = ks.utils.to_categorical(y_train, len(np.unique(y_train)))
```

[58]:
```
# printing to see if shapes are correct
print(f'X_train shape: {X_train.shape} \n'
      f'X_test shape: {X_test.shape} \n'
      f'y_train shape: {y_train.shape}')
```

```
X_train shape: (59500, 28, 28, 1)
X_test shape: (10500, 28, 28, 1)
y_train shape: (59500, 10)
```

6.3 Task 2.2: Visualize the dataset

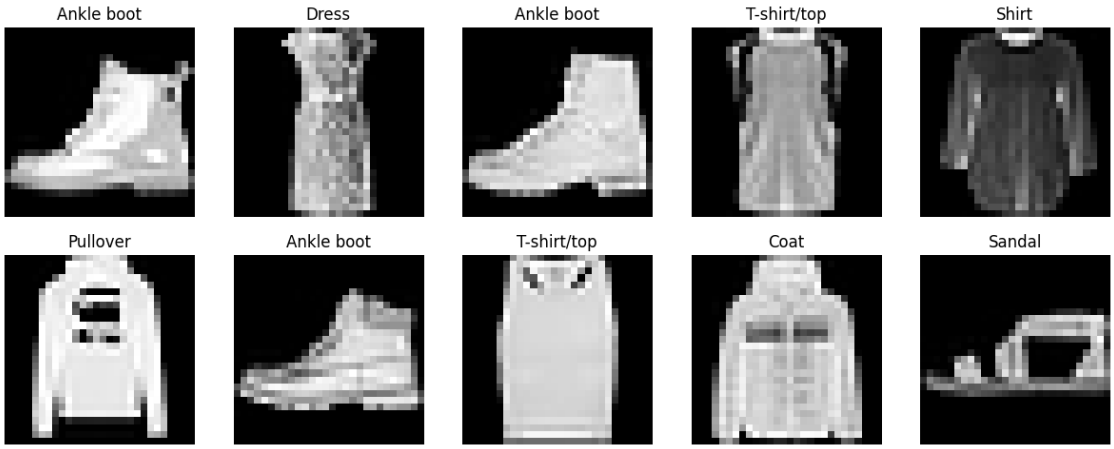
Plot a few samples images and the distrbution of classes in the data.

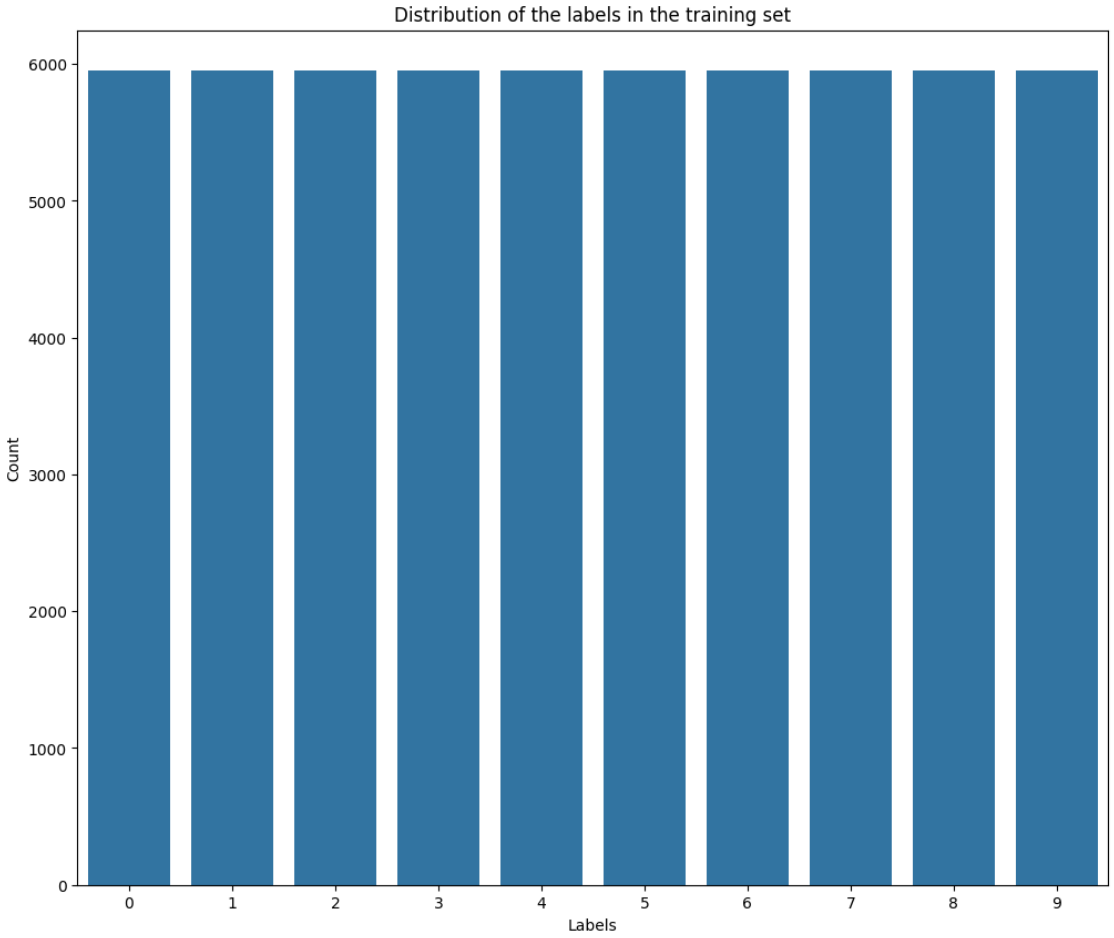
[59]:
```
# ploting a few sample images
plt.figure(figsize=(15,15))

for i in range(10):
    plt.subplot(5,5, i+1)
    plt.imshow(X_train[i+1], cmap='gray')
    plt.title(labels[np.argmax(y_train[i+1])])
    plt.axis('off')
```

```
# ploting the distribution of the classes
plt.figure(figsize=(12,10))
sns.countplot(x=np.argmax(y_train,axis=1))
plt.title('Distribution of the labels in the training set')
plt.xlabel('Labels')
plt.ylabel('Count')
```

[59]: Text(0, 0.5, 'Count')





6.4 Task 2.3: Build a CNN for classifying the Fashion-MNIST dataset

Build a CNN model - Experiment with different, Kernel sizes, stride, type/number of layers. - Don't overcomplicate and use it as grounds for discussion. - Tips, when you make changes to improve the model, save some earlier iterations. - Only one model score higher than beat me.

6.5 2 conv and avg. pooling

[60]:
```
# build the model
model_v1 = ks.Sequential([
    ks.layers.Conv2D(filters=6, kernel_size=(3, 3), padding='same',␣
 ↪name='Conv1', activation='relu'),
    ks.layers.AvgPool2D(pool_size=(2, 2), strides=(2, 2), name='Pool1'),
    ks.layers.Conv2D(filters=16, kernel_size=(3, 3), padding='valid',␣
 ↪name='Conv2', activation='relu'),
    ks.layers.AvgPool2D(pool_size=(2, 2), strides=(2, 2), name='Pool2'),
    ks.layers.Flatten(),  # flatten before giving to nn
    ks.layers.Dense(units=1024, name='FC1', activation='relu'),
```

```
    ks.layers.Dense(units=128, name='FC2', activation='relu'),
    ks.layers.Dense(units=10, name='Output', activation='softmax'),
])

# don't need to specify batch size
model_v1.build(input_shape=(None, 28, 28, 1))

model_v1.compile(
    optimizer=ks.optimizers.Adam(learning_rate=0.001),
    loss=ks.losses.CategoricalCrossentropy(),
    metrics=['accuracy', f1score]
)

model_v1.summary()
```

```
Model: "sequential_10"
```

Layer (type)	Output Shape	Param #
Conv1 (Conv2D)	(None, 28, 28, 6)	60
Pool1 (AveragePooling2D)	(None, 14, 14, 6)	0
Conv2 (Conv2D)	(None, 12, 12, 16)	880
Pool2 (AveragePooling2D)	(None, 6, 6, 16)	0
flatten_10 (Flatten)	(None, 576)	0
FC1 (Dense)	(None, 1024)	590,848
FC2 (Dense)	(None, 128)	131,200
Output (Dense)	(None, 10)	1,290

```
Total params: 724,278 (2.76 MB)

Trainable params: 724,278 (2.76 MB)

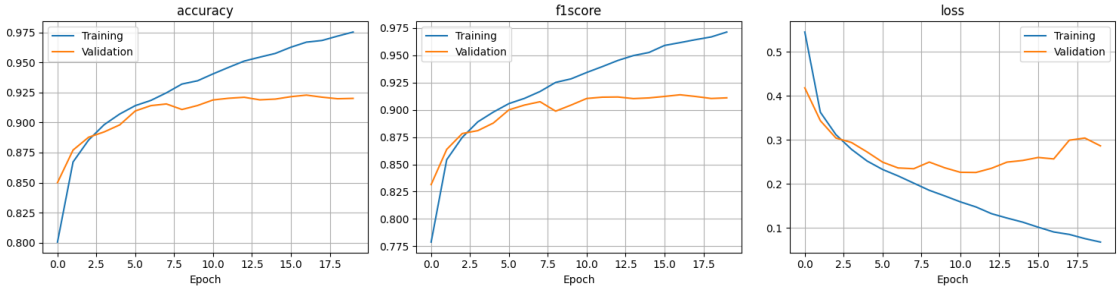
Non-trainable params: 0 (0.00 B)
```

[61]:
```
# trian the model
history_fashion_v1 = model_v1.fit(X_train, y_train, batch_size=64, epochs=20,␣
↪validation_split=1/6)
```

```
Epoch 1/20
775/775             6s 7ms/step -
accuracy: 0.8005 - f1score: 0.7788 - loss: 0.5446 - val_accuracy: 0.8500 -
val_f1score: 0.8315 - val_loss: 0.4178
Epoch 2/20
775/775             5s 7ms/step -
accuracy: 0.8672 - f1score: 0.8543 - loss: 0.3625 - val_accuracy: 0.8771 -
val_f1score: 0.8638 - val_loss: 0.3433
Epoch 3/20
775/775             5s 7ms/step -
accuracy: 0.8854 - f1score: 0.8748 - loss: 0.3116 - val_accuracy: 0.8878 -
val_f1score: 0.8783 - val_loss: 0.3040
Epoch 4/20
775/775             5s 6ms/step -
accuracy: 0.8982 - f1score: 0.8890 - loss: 0.2783 - val_accuracy: 0.8922 -
val_f1score: 0.8810 - val_loss: 0.2938
Epoch 5/20
775/775             5s 6ms/step -
accuracy: 0.9071 - f1score: 0.8980 - loss: 0.2518 - val_accuracy: 0.8981 -
val_f1score: 0.8879 - val_loss: 0.2724
Epoch 6/20
775/775             5s 6ms/step -
accuracy: 0.9141 - f1score: 0.9058 - loss: 0.2326 - val_accuracy: 0.9097 -
val_f1score: 0.9000 - val_loss: 0.2492
Epoch 7/20
775/775             5s 6ms/step -
accuracy: 0.9184 - f1score: 0.9106 - loss: 0.2179 - val_accuracy: 0.9141 -
val_f1score: 0.9044 - val_loss: 0.2360
Epoch 8/20
775/775             5s 7ms/step -
accuracy: 0.9248 - f1score: 0.9169 - loss: 0.2016 - val_accuracy: 0.9155 -
val_f1score: 0.9075 - val_loss: 0.2343
Epoch 9/20
775/775             5s 6ms/step -
accuracy: 0.9321 - f1score: 0.9252 - loss: 0.1853 - val_accuracy: 0.9109 -
val_f1score: 0.8988 - val_loss: 0.2494
Epoch 10/20
775/775             5s 6ms/step -
accuracy: 0.9348 - f1score: 0.9285 - loss: 0.1724 - val_accuracy: 0.9142 -
val_f1score: 0.9044 - val_loss: 0.2363
Epoch 11/20
775/775             5s 6ms/step -
accuracy: 0.9405 - f1score: 0.9343 - loss: 0.1590 - val_accuracy: 0.9188 -
```

```
val_f1score: 0.9105 - val_loss: 0.2261
Epoch 12/20
775/775            5s 6ms/step -
accuracy: 0.9460 - f1score: 0.9397 - loss: 0.1475 - val_accuracy: 0.9202 -
val_f1score: 0.9117 - val_loss: 0.2257
Epoch 13/20
775/775            5s 6ms/step -
accuracy: 0.9512 - f1score: 0.9453 - loss: 0.1322 - val_accuracy: 0.9210 -
val_f1score: 0.9118 - val_loss: 0.2353
Epoch 14/20
775/775            5s 6ms/step -
accuracy: 0.9545 - f1score: 0.9498 - loss: 0.1222 - val_accuracy: 0.9189 -
val_f1score: 0.9104 - val_loss: 0.2493
Epoch 15/20
775/775            5s 6ms/step -
accuracy: 0.9576 - f1score: 0.9526 - loss: 0.1130 - val_accuracy: 0.9195 -
val_f1score: 0.9110 - val_loss: 0.2530
Epoch 16/20
775/775            5s 7ms/step -
accuracy: 0.9627 - f1score: 0.9591 - loss: 0.1015 - val_accuracy: 0.9215 -
val_f1score: 0.9123 - val_loss: 0.2598
Epoch 17/20
775/775            5s 7ms/step -
accuracy: 0.9670 - f1score: 0.9617 - loss: 0.0907 - val_accuracy: 0.9229 -
val_f1score: 0.9138 - val_loss: 0.2565
Epoch 18/20
775/775            5s 7ms/step -
accuracy: 0.9683 - f1score: 0.9644 - loss: 0.0851 - val_accuracy: 0.9211 -
val_f1score: 0.9123 - val_loss: 0.2991
Epoch 19/20
775/775            5s 7ms/step -
accuracy: 0.9720 - f1score: 0.9669 - loss: 0.0756 - val_accuracy: 0.9198 -
val_f1score: 0.9105 - val_loss: 0.3039
Epoch 20/20
775/775            6s 7ms/step -
accuracy: 0.9754 - f1score: 0.9714 - loss: 0.0677 - val_accuracy: 0.9201 -
val_f1score: 0.9110 - val_loss: 0.2862
```

[62]:
```
plot_training_history(history_fashion_v1)
```



6.6 Standard, 2 conv and max. pooling

[63]:
```
# test the same model only with maxpoolin instead of average, to extract␣
 ↪features better
# build the model
model_v2 = ks.Sequential([
    ks.layers.Conv2D(filters=6, kernel_size=(3, 3), padding='same',␣
 ↪name='Conv1', activation='relu'),
    ks.layers.MaxPool2D(pool_size=(2, 2), strides=(2, 2), name='Pool1'),
    ks.layers.Conv2D(filters=16, kernel_size=(3, 3), padding='valid',␣
 ↪name='Conv2', activation='relu'),
    ks.layers.MaxPool2D(pool_size=(2, 2), strides=(2, 2), name='Pool2'),
    ks.layers.Flatten(),  # flatten before giving to nn
    ks.layers.Dense(units=1024, name='FC1', activation='relu'),
    ks.layers.Dense(units=128, name='FC2', activation='relu'),
    ks.layers.Dense(units=10, name='Output', activation='softmax'),
])

# don't need to specify batch size
model_v2.build(input_shape=(None, 28, 28, 1))

model_v2.compile(
    optimizer=ks.optimizers.Adam(),
    loss=ks.losses.CategoricalCrossentropy(),
    metrics=['accuracy', f1score]
)

model_v2.summary()
```

```
Model: "sequential_11"
```

Layer (type)	Output Shape	Param #
Conv1 (Conv2D)	(None, 28, 28, 6)	60

```
 Pool1 (MaxPooling2D)            (None, 14, 14, 6)                0

 Conv2 (Conv2D)                  (None, 12, 12, 16)             880

 Pool2 (MaxPooling2D)            (None, 6, 6, 16)                 0

 flatten_11 (Flatten)            (None, 576)                      0

 FC1 (Dense)                     (None, 1024)               590,848

 FC2 (Dense)                     (None, 128)                131,200

 Output (Dense)                  (None, 10)                   1,290


Total params: 724,278 (2.76 MB)

Trainable params: 724,278 (2.76 MB)

Non-trainable params: 0 (0.00 B)
```

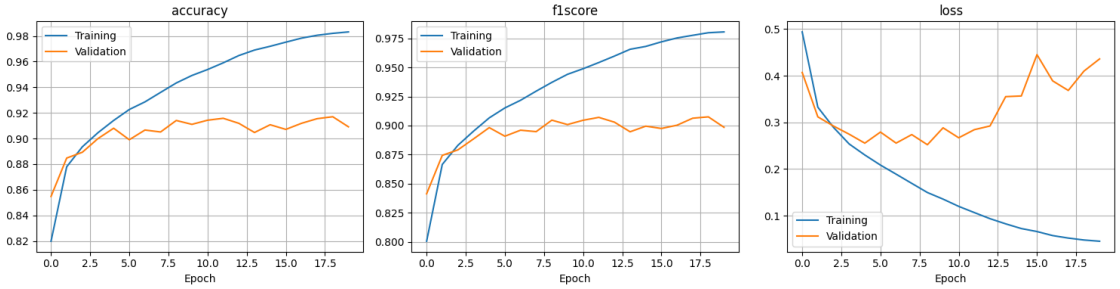
[64]:
```
history_fashion_v2 = model_v2.fit(X_train, y_train, batch_size=64, epochs=20,␣
↪validation_split=1/6)
```

```
Epoch 1/20
775/775             6s 7ms/step -
accuracy: 0.8198 - f1score: 0.8004 - loss: 0.4937 - val_accuracy: 0.8548 -
val_f1score: 0.8413 - val_loss: 0.4069
Epoch 2/20
775/775             5s 6ms/step -
accuracy: 0.8779 - f1score: 0.8665 - loss: 0.3324 - val_accuracy: 0.8847 -
val_f1score: 0.8743 - val_loss: 0.3118
Epoch 3/20
775/775             5s 6ms/step -
accuracy: 0.8935 - f1score: 0.8830 - loss: 0.2892 - val_accuracy: 0.8893 -
val_f1score: 0.8791 - val_loss: 0.2922
Epoch 4/20
775/775             5s 6ms/step -
accuracy: 0.9046 - f1score: 0.8952 - loss: 0.2538 - val_accuracy: 0.9000 -
val_f1score: 0.8885 - val_loss: 0.2747
Epoch 5/20
775/775             5s 6ms/step -
accuracy: 0.9141 - f1score: 0.9066 - loss: 0.2302 - val_accuracy: 0.9080 -
val_f1score: 0.8982 - val_loss: 0.2555
```

```
Epoch 6/20
775/775            5s 6ms/step -
accuracy: 0.9227 - f1score: 0.9152 - loss: 0.2085 - val_accuracy: 0.8991 -
val_f1score: 0.8908 - val_loss: 0.2792
Epoch 7/20
775/775            5s 6ms/step -
accuracy: 0.9286 - f1score: 0.9217 - loss: 0.1891 - val_accuracy: 0.9066 -
val_f1score: 0.8960 - val_loss: 0.2556
Epoch 8/20
775/775            5s 6ms/step -
accuracy: 0.9360 - f1score: 0.9296 - loss: 0.1694 - val_accuracy: 0.9051 -
val_f1score: 0.8948 - val_loss: 0.2738
Epoch 9/20
775/775            6s 7ms/step -
accuracy: 0.9434 - f1score: 0.9372 - loss: 0.1498 - val_accuracy: 0.9141 -
val_f1score: 0.9046 - val_loss: 0.2521
Epoch 10/20
775/775            5s 7ms/step -
accuracy: 0.9492 - f1score: 0.9441 - loss: 0.1357 - val_accuracy: 0.9111 -
val_f1score: 0.9008 - val_loss: 0.2883
Epoch 11/20
775/775            6s 7ms/step -
accuracy: 0.9539 - f1score: 0.9489 - loss: 0.1201 - val_accuracy: 0.9143 -
val_f1score: 0.9046 - val_loss: 0.2670
Epoch 12/20
775/775            6s 7ms/step -
accuracy: 0.9591 - f1score: 0.9542 - loss: 0.1068 - val_accuracy: 0.9158 -
val_f1score: 0.9070 - val_loss: 0.2844
Epoch 13/20
775/775            5s 6ms/step -
accuracy: 0.9648 - f1score: 0.9598 - loss: 0.0938 - val_accuracy: 0.9119 -
val_f1score: 0.9029 - val_loss: 0.2923
Epoch 14/20
775/775            5s 7ms/step -
accuracy: 0.9691 - f1score: 0.9656 - loss: 0.0827 - val_accuracy: 0.9047 -
val_f1score: 0.8946 - val_loss: 0.3551
Epoch 15/20
775/775            5s 7ms/step -
accuracy: 0.9720 - f1score: 0.9681 - loss: 0.0726 - val_accuracy: 0.9108 -
val_f1score: 0.8995 - val_loss: 0.3565
Epoch 16/20
775/775            5s 7ms/step -
accuracy: 0.9752 - f1score: 0.9720 - loss: 0.0662 - val_accuracy: 0.9071 -
val_f1score: 0.8975 - val_loss: 0.4450
Epoch 17/20
775/775            5s 7ms/step -
accuracy: 0.9784 - f1score: 0.9754 - loss: 0.0576 - val_accuracy: 0.9119 -
val_f1score: 0.9003 - val_loss: 0.3889
```

```
Epoch 18/20
775/775             5s 6ms/step -
accuracy: 0.9806 - f1score: 0.9777 - loss: 0.0523 - val_accuracy: 0.9155 -
val_f1score: 0.9063 - val_loss: 0.3683
Epoch 19/20
775/775             5s 6ms/step -
accuracy: 0.9821 - f1score: 0.9799 - loss: 0.0481 - val_accuracy: 0.9170 -
val_f1score: 0.9075 - val_loss: 0.4097
Epoch 20/20
775/775             5s 6ms/step -
accuracy: 0.9832 - f1score: 0.9806 - loss: 0.0455 - val_accuracy: 0.9091 -
val_f1score: 0.8985 - val_loss: 0.4359
```

[65]:
```
plot_training_history(history_fashion_v2)
```



6.7 Standard, 4 conv and max. pooling and regularization (batch normalization and Dropout)

[66]:
```
# build the model
model_v3 = ks.Sequential([
    ks.layers.Conv2D(filters=8, kernel_size=(3, 3), padding='same',␣
 ↪name='Conv1', activation='relu'),
    ks.layers.BatchNormalization(name='BN1'),
    ks.layers.Conv2D(filters=16, kernel_size=(3, 3), padding='same',␣
 ↪name='Conv2', activation='relu'),
    ks.layers.BatchNormalization(name='BN2'),
    ks.layers.MaxPool2D(pool_size=(2, 2), strides=(2, 2), name='Pool1'),
    ks.layers.Dropout(0.25, name='Drop1'),
    ks.layers.Conv2D(filters=32, kernel_size=(3, 3), padding='same',␣
 ↪name='Conv3', activation='relu'),
    ks.layers.BatchNormalization(name='BN3'),
    ks.layers.Conv2D(filters=64, kernel_size=(3, 3), padding='same',␣
 ↪name='Conv4', activation='relu'),
    ks.layers.BatchNormalization(name='BN4'),
    ks.layers.MaxPool2D(pool_size=(2, 2), strides=(2, 2), name='Pool2'),
```

```
    ks.layers.Dropout(0.25, name='Drop2'),
    ks.layers.Flatten(),  # flatten before giving to nn
    ks.layers.Dense(units=512, name='FC1', activation='relu'),
    ks.layers.Dropout(0.5, name='Drop3'),
    ks.layers.Dense(units=128, name='FC2', activation='relu'),
    ks.layers.Dropout(0.5, name='Drop4'),
    ks.layers.Dense(units=10, name='Output', activation='softmax'),
])

# don't need to specify batch size
model_v3.build(input_shape=(None, 28, 28, 1))

model_v3.compile(
    optimizer=ks.optimizers.Adam(),
    loss=ks.losses.CategoricalCrossentropy(),
    metrics=['accuracy', f1score]
)

model_v3.summary()
```

```
Model: "sequential_12"
```

Layer (type)	Output Shape	Param #
Conv1 (Conv2D)	(None, 28, 28, 8)	80
BN1 (BatchNormalization)	(None, 28, 28, 8)	32
Conv2 (Conv2D)	(None, 28, 28, 16)	1,168
BN2 (BatchNormalization)	(None, 28, 28, 16)	64
Pool1 (MaxPooling2D)	(None, 14, 14, 16)	0
Drop1 (Dropout)	(None, 14, 14, 16)	0
Conv3 (Conv2D)	(None, 14, 14, 32)	4,640
BN3 (BatchNormalization)	(None, 14, 14, 32)	128
Conv4 (Conv2D)	(None, 14, 14, 64)	18,496
BN4 (BatchNormalization)	(None, 14, 14, 64)	256
Pool2 (MaxPooling2D)	(None, 7, 7, 64)	0

Drop2 (Dropout)	(None, 7, 7, 64)	0
flatten_12 (Flatten)	(None, 3136)	0
FC1 (Dense)	(None, 512)	1,606,144
Drop3 (Dropout)	(None, 512)	0
FC2 (Dense)	(None, 128)	65,664
Drop4 (Dropout)	(None, 128)	0
Output (Dense)	(None, 10)	1,290

Total params: 1,697,962 (6.48 MB)

Trainable params: 1,697,722 (6.48 MB)

Non-trainable params: 240 (960.00 B)

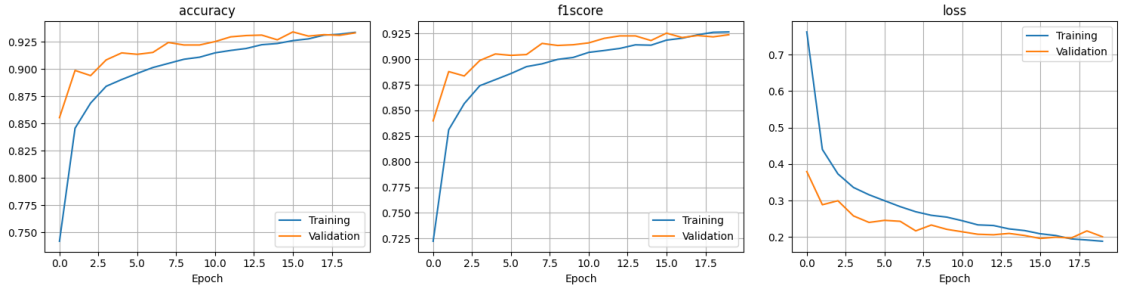
[67]:
```
history_fashion_v3 = model_v3.fit(X_train, y_train, batch_size=64, epochs=20,␣
↪validation_split=1/6)
```

```
Epoch 1/20
775/775             26s 31ms/step -
accuracy: 0.7417 - f1score: 0.7224 - loss: 0.7620 - val_accuracy: 0.8553 -
val_f1score: 0.8398 - val_loss: 0.3796
Epoch 2/20
775/775             21s 27ms/step -
accuracy: 0.8456 - f1score: 0.8311 - loss: 0.4402 - val_accuracy: 0.8987 -
val_f1score: 0.8878 - val_loss: 0.2887
Epoch 3/20
775/775             21s 27ms/step -
accuracy: 0.8687 - f1score: 0.8566 - loss: 0.3732 - val_accuracy: 0.8939 -
val_f1score: 0.8836 - val_loss: 0.2997
Epoch 4/20
775/775             22s 29ms/step -
accuracy: 0.8841 - f1score: 0.8741 - loss: 0.3361 - val_accuracy: 0.9083 -
val_f1score: 0.8987 - val_loss: 0.2581
Epoch 5/20
775/775             22s 28ms/step -
accuracy: 0.8903 - f1score: 0.8800 - loss: 0.3160 - val_accuracy: 0.9148 -
val_f1score: 0.9050 - val_loss: 0.2403
Epoch 6/20
```

```
775/775            23s 29ms/step -
accuracy: 0.8960 - f1score: 0.8859 - loss: 0.2998 - val_accuracy: 0.9135 -
val_f1score: 0.9037 - val_loss: 0.2460
Epoch 7/20
775/775            22s 28ms/step -
accuracy: 0.9013 - f1score: 0.8927 - loss: 0.2836 - val_accuracy: 0.9152 -
val_f1score: 0.9045 - val_loss: 0.2434
Epoch 8/20
775/775            24s 30ms/step -
accuracy: 0.9051 - f1score: 0.8954 - loss: 0.2695 - val_accuracy: 0.9244 -
val_f1score: 0.9153 - val_loss: 0.2172
Epoch 9/20
775/775            25s 32ms/step -
accuracy: 0.9089 - f1score: 0.8997 - loss: 0.2598 - val_accuracy: 0.9221 -
val_f1score: 0.9133 - val_loss: 0.2332
Epoch 10/20
775/775            24s 31ms/step -
accuracy: 0.9108 - f1score: 0.9016 - loss: 0.2548 - val_accuracy: 0.9221 -
val_f1score: 0.9140 - val_loss: 0.2214
Epoch 11/20
775/775            24s 30ms/step -
accuracy: 0.9148 - f1score: 0.9066 - loss: 0.2449 - val_accuracy: 0.9252 -
val_f1score: 0.9158 - val_loss: 0.2146
Epoch 12/20
775/775            26s 34ms/step -
accuracy: 0.9170 - f1score: 0.9084 - loss: 0.2336 - val_accuracy: 0.9295 -
val_f1score: 0.9203 - val_loss: 0.2078
Epoch 13/20
775/775            27s 34ms/step -
accuracy: 0.9188 - f1score: 0.9104 - loss: 0.2318 - val_accuracy: 0.9307 -
val_f1score: 0.9227 - val_loss: 0.2065
Epoch 14/20
775/775            24s 31ms/step -
accuracy: 0.9223 - f1score: 0.9140 - loss: 0.2226 - val_accuracy: 0.9311 -
val_f1score: 0.9227 - val_loss: 0.2102
Epoch 15/20
775/775            24s 31ms/step -
accuracy: 0.9234 - f1score: 0.9136 - loss: 0.2180 - val_accuracy: 0.9268 -
val_f1score: 0.9181 - val_loss: 0.2045
Epoch 16/20
775/775            25s 33ms/step -
accuracy: 0.9261 - f1score: 0.9185 - loss: 0.2093 - val_accuracy: 0.9341 -
val_f1score: 0.9254 - val_loss: 0.1964
Epoch 17/20
775/775            24s 31ms/step -
accuracy: 0.9277 - f1score: 0.9204 - loss: 0.2044 - val_accuracy: 0.9302 -
val_f1score: 0.9211 - val_loss: 0.1999
Epoch 18/20
```

```
775/775              25s 32ms/step -
accuracy: 0.9311 - f1score: 0.9237 - loss: 0.1952 - val_accuracy: 0.9315 -
val_f1score: 0.9230 - val_loss: 0.1977
Epoch 19/20
775/775              26s 33ms/step -
accuracy: 0.9320 - f1score: 0.9262 - loss: 0.1922 - val_accuracy: 0.9309 -
val_f1score: 0.9217 - val_loss: 0.2171
Epoch 20/20
775/775              25s 33ms/step -
accuracy: 0.9336 - f1score: 0.9265 - loss: 0.1888 - val_accuracy: 0.9331 -
val_f1score: 0.9238 - val_loss: 0.2012
```

[68]:
```
plot_training_history(history_fashion_v3)
```



7 Task 2.5: Fashion-MNIST Discussion

7.0.1 Task 3.5.1

- Feel Free to experiment on architecture
- Comment on the choice of layers and hyperparameters.
  - Did you find some different results in changing a hyperparameter or add/remove a layer.

We trained three different models, changing one major thing in each.

Model	Architecture	Best validation loss	Best validation acc.
V1	2 conv layers (6/16 filters), average pooling	0.2376 (epoch 7)	0.9211 (epoch 19)
V2	2 conv layers (6/16 filters), max pooling	0.2493 (epoch 7)	0.9150 (epoch 11)

Model	Architecture	Best validation loss	Best validation acc.
V3	4 conv layers (8/16/32/64 filters) with batch normalization after each, max pooling (stride 2) after every second conv layer, dropout 0.25 after each pooling layer, and dense layers (512/128) with dropout 0.5	0.1895 (epoch 19)	0.9377 (epoch 19)

In V1 we used average pooling. In V2 we switched to max pooling, expecting it to capture edges better and give higher accuracy. Both models started overfitting after roughly 5 epochs. In V3 we therefore tried a larger network, but also added more regularization in the form of batch normalization and dropout.

Model V3 clearly gave the best results and did not overfit. It could also have been trained for more epochs.